

Introducción a la Ingeniería de Software. ¿Qué es?

La Ingeniería de Software es la disciplina encargada de todos los aspectos relacionados a la producción de software, desde la especificación y diseño hasta su operación, evolución y mantenimiento.

Existen varias definiciones clave que la caracterizan:

- **Definición oficial de la IEEE (SWEBOK):** Es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software; es decir, la aplicación de los principios de la ingeniería al desarrollo informático.
- **Definición de Sommerville:** Una disciplina centrada en todos los aspectos de la producción del software.
- **Definición de Parnas:** La construcción por parte de múltiples personas de software con múltiples versiones ("*multi-person construction of multi-version software*").

¿Qué se entiende por Software?

El software no es sólo código fuente; es el conjunto de **programas informáticos, datos, procedimientos, reglas y toda la documentación asociada** (manuales, documentos técnicos, datos de prueba y archivos de configuración) requeridos para definir, desarrollar y mantener los sistemas.

Una definición más integradora lo describe como **conocimiento representado en distintos niveles de abstracción**.

Origen: La Crisis del Software

La Ingeniería de Software nace como respuesta a la **crisis del software**, término acuñado en la conferencia de la OTAN en 1968. Dicha crisis evidenció las grandes dificultades para generar software libre de defectos, dentro de los plazos y presupuestos estimados, dejando en claro que **saber programar no es suficiente** cuando se afrontan proyectos complejos.

Actividades Fundamentales

El proceso de ingeniería de software contempla cuatro actividades:

1. **Especificación:** Definir lo que el sistema debe hacer y sus restricciones.
2. **Desarrollo:** Diseño de la arquitectura y programación del software.

3. **Validación:** Verificación y pruebas para asegurar que cumpla con los requerimientos del cliente.
4. **Evolución:** Mantenimiento y adaptación del sistema ante cambios del negocio o mercado.

Disciplinas de la Ingeniería de Software

Las actividades de la Ingeniería de Software se estructuran en tres grandes grupos:

- **Disciplinas Técnicas:** Enfocadas directamente en el producto (Requerimientos, Análisis y Diseño, Construcción, Pruebas y Despliegue).
- **Disciplinas de Gestión:** Orientadas a la planificación, monitoreo y control del proyecto de desarrollo.
- **Disciplinas de Soporte:** Transversales al ciclo de vida para garantizar la calidad e integridad del producto (Gestión de Configuración - SCM, Aseguramiento de Calidad - QA y Métricas).

Estado Actual y Antecedentes. La Crisis del Software.

Antecedentes: La Crisis del Software

El término **crisis del software** fue acuñado originalmente durante la histórica conferencia de la OTAN en 1968 por Friedrich Bauer. Este concepto engloba las severas dificultades que padecía la industria para construir software libre de defectos, comprensible, fácil de verificar y entregado a tiempo y dentro de los costos presupuestados.

Causas Principales de la Crisis

- **Evolución del Hardware vs. Software:** La llegada de los circuitos integrados provocó un salto en la capacidad del hardware, permitiendo diseñar computadoras más potentes y demandando sistemas mucho más complejos. Sin embargo, la construcción de software no pudo acompañar ese ritmo, ya que el software es un producto abstracto, intangible y fruto del esfuerzo intelectual humano.

- **Subestimación de la complejidad:** Se asumió erróneamente que el desarrollo de grandes sistemas era equivalente a programar a pequeña escala.
 - **Falta de Disciplinas de Ingeniería:** En sus inicios, la programación carecía de metodologías estructuradas, estándares de calidad y procesos de gestión.
 - **Consecuencias Históricas:** Esto derivó en productos de mala calidad, difíciles de escalar, con sobrecostos y retrasos sistemáticos.
-

Estado Actual de la Ingeniería de Software

A pesar del surgimiento de la Ingeniería de Software como disciplina y de la evolución de múltiples marcos metodológicos, **muchos proyectos de software continúan fallando en la actualidad**. El estado actual de la industria refleja dos factores determinantes en estas fallas:

1. **Demanda Creciente:** Las necesidades del mercado exigen construir sistemas cada vez más grandes, integrados y complejos en plazos de entrega reducidos. Los métodos tradicionales muchas veces no brindan la velocidad requerida, lo que obliga a probar constantemente nuevas técnicas de desarrollo y gestión.
2. **Bajas Expectativas y Falta de Métodos:** Muchas organizaciones aún no aplican metodologías y métodos formales de ingeniería. Esto genera que tanto desarrolladores como clientes se conforman con que el software simplemente "funcione", pasando por alto aspectos de confiabilidad, mantenibilidad y valor agregado para el negocio.

No Silver Bullet

1. Introducción: la metáfora del hombre lobo

Brooks compara los proyectos de software con los hombres lobo: parecen inofensivos y controlables, pero pueden transformarse repentinamente en monstruos de plazos

incumplidos, presupuestos desbordados y productos defectuosos. Ante esto, la industria pide a gritos una "bala de plata": algo que haga caer los costos del software tan rápido como cayeron los del hardware. Brooks adelanta su tesis central: **no existe, ni existirá en los próximos diez años, ningún desarrollo único** capaz de producir una mejora de un orden de magnitud en productividad, confiabilidad o simplicidad. Aclara que esto es escepticismo, no pesimismo: sí habrá progreso, pero gradual y trabajoso, no mágico .

2. ¿Tiene que ser difícil? – Las dificultades esenciales

Aquí Brooks retoma la distinción aristotélica entre **esencia** (las dificultades inherentes a la naturaleza del software) y **accidentes** (las dificultades que hoy acompañan su producción, pero que no son inherentes). Sostiene que lo verdaderamente difícil de construir software es la especificación, el diseño y las pruebas de esa estructura conceptual abstracta, no el trabajo de representarla en un lenguaje. Si esto es así, concluye, construir software siempre será difícil por naturaleza.

Dificultades Esenciales

Son los problemas **inherentes a la naturaleza misma del software**. Corresponden a la construcción conceptual del sistema abstracto (el diseño de algoritmos, estructuras de datos, relaciones e invocaciones funcionales). Construir el modelo mental correcto de lo que debe hacer el sistema representa la parte más difícil y propensa a errores.

Identifica **cuatro propiedades esenciales** del software:

- **Complejidad:** el software se vuelve más complejo en función de su tamaño. Esta complejidad crece de forma no lineal al escalar el sistema y no se puede simplificar mediante abstracciones sin perder la esencia del problema.
- **Conformidad:** El software debe adaptarse y ajustarse arbitrariamente a interfaces humanas, regulaciones, hardware y sistemas heredados diseñados por distintas personas o instituciones.
- **Mutabilidad:** el software está sometido a una presión constante de cambio. Al ser un producto intangible se lo percibe como maleable. Las organizaciones prefieren modificar software antes que cambiar procesos de negocio o componentes físicos.
- **Invisibilidad:** el software es intangible e imposible de visualizar espacial o geométricamente. Los diagramas o grafos representan sólo perspectivas parciales que dificultan el diseño y la representación de los sistemas.

Dificultades Accidentales

Son obstáculos o limitaciones relacionadas al **proceso de desarrollo, los lenguajes, las herramientas o tecnologías** utilizadas para representar una construcción conceptual en el mundo real. No forman parte de la esencia del problema, y por lo tanto, pueden mitigarse mediante mejoras técnicas y avances tecnológicos.

3. Los avances pasados resolvieron dificultades accidentales, no esenciales

Brooks revisa los tres grandes saltos de productividad del pasado y muestra que **todos atacaron accidentes, no la esencia**:

- **Lenguajes de alto nivel**: liberaron al programador de la complejidad accidental de las máquinas (detalles técnicos de bajo nivel). Sin embargo, el programador aún debe comprender y diseñar una solución al problema.
- **Tiempo compartido (time-sharing)**: aumentó la productividad al reducir los tiempos de espera del procesamiento por lotes, permitiendo una interacción inmediata con el sistema y evitando que el programador perdiera el hilo de pensamiento. Su límite es el umbral de la percepción humana; más allá de eso no hay beneficio adicional.
- **Entornos de desarrollo e IDEs**: aumentaron la productividad al reducir la complejidad accidental derivada del uso de herramientas independientes. La integración de bibliotecas, formatos de archivo unificados y mecanismos de comunicación entre programas permite trabajar de forma más fluida. Sin embargo, no eliminan la complejidad esencial de comprender y resolver el problema.

4. Esperanzas de "balas de plata" — y por qué Brooks las descarta

Analiza una por una las tecnologías que en su momento se promocionaban como posibles soluciones "mágicas", evaluando si atacan la esencia o solo accidentes:

- **Lenguajes de alto nivel**: Liberaron a los programadores del detalle técnico del lenguaje ensamblador y de máquina pero no ataca la esencia.

- **Programación orientada a objetos:** El POO permite representar mejor el dominio mediante objetos, clases, jerarquías, etc. Si bien facilita expresar el diseño de la solución, no elimina la complejidad de crear ese diseño.
- **Inteligencia artificial:** Una IA puede ayudar a implementar una solución, pero alguien tiene que comprender el problema y determinar qué se quiere lograr.
- **Sistemas expertos:** Permite que personas inexpertas utilicen el conocimiento de especialistas, pero el problema radica en conseguir esos expertos, que estos logren articular y expresar su conocimiento y convertir ese conocimiento en reglas. Tiene cierto potencial pero no constituye una “bala de plata”.
- **Programación automática:** Brooks señala que muchas veces esto simplemente significa programar utilizando un lenguaje de mayor nivel. Puede funcionar bien en dominios muy específicos, como problemas simples, acotados y formales, pero en sistemas complejos no sirven de mucho.
- **Programación gráfica:** La idea de que “Si podemos representar gráficamente el software será más fácil comprenderlo”, es cuestionada por Brooks, ya que el software tiene múltiples dimensiones y relaciones simultáneas. Un diagrama podría ser útil para una parte del sistema, pero tratar de representar todo el sistema gráficamente puede resultar incluso más complejo que el propio sistema.
- **Verificación formal de programas:** es valiosa para sistemas críticos, pero no ahorra trabajo (verificar es carísimo) y, sobre todo, solo garantiza que el programa cumple su especificación — no resuelve el problema de definir bien esa especificación. Si definimos mal el problema, podemos tener un programa perfectamente verificado que resuelve el problema equivocado.
- **Entornos y herramientas, y estaciones de trabajo más potentes:** Los IDE, herramientas integradas, mejores procesadores, más memoria, etc., sí mejoran la productividad. Pero Brooks sostiene que los grandes beneficios de este tipo de mejoras ya fueron obtenidos. El verdadero cuello de botella del desarrollador puede estar en comprender el problema, diseñar la solución y tomar decisiones, no en cuánto tarda la computadora en compilar.

5. Ataques prometedores sobre la esencia conceptual

Para lograr avances reales sobre las dificultades esenciales, Brooks propone enfoques que reducen la necesidad de construir conceptos desde cero

- **Comprar en vez de construir:** Reutilizar software comercial o componentes existentes evita volver a crear la lógica conceptual desde cero. Además, adquirir software ya existente es más barato que desarrollarlo a medida, porque el costo del software está en su desarrollo, no en replicarlo.
 - **Refinamiento de requerimientos y prototipado rápido:** Brooks sostiene que una de las partes más difíciles del desarrollo es determinar exactamente qué se debe construir, ya que los clientes pueden no conocer o expresar con precisión sus necesidades. El prototipado permite mostrar versiones iniciales del sistema y obtener feedback del cliente, refinando los requerimientos de manera iterativa en lugar de intentar definirlos completamente desde el inicio.
 - **Desarrollo incremental – "cultivar" en vez de "construir" software:** Hacer crecer el sistema añadiendo capas progresivas en lugar de intentar diseñarlo de una sola vez
 - **Cultivar grandes diseñadores:** Fomentar el talento y la integridad conceptual del diseño en las mentes de los desarrolladores.
-

Resumen:

La evolución tecnológica (como lenguajes de alto nivel, entornos integrados e incluso la inteligencia artificial) ha logrado eliminar gran parte de la **complejidad accidental**. No obstante, la **complejidad esencial** del software sigue siendo el cuello de botella central que ninguna "bala de plata" tecnológica ha podido resolver de forma automática.

Disciplinas que conforman la Ingeniería de Software

Las disciplinas que conforman la Ingeniería de Software se estructuran en tres grandes categorías.

1. Disciplinas Técnicas

Son el conjunto de actividades enfocadas directamente a la construcción y entrega del software como producto final.

- **Toma de Requerimientos:** Captura (elicitación), análisis y especificación de las necesidades del cliente, traduciéndolas en especificaciones formales (ERS) o Historias de Usuario (US)
 - **Análisis y Diseño:** Definición de la arquitectura, estructura de datos, patrones de diseño e interfaces de usuario. Establece el modelo y la solución conceptual previo a la implementación.
 - **Construcción e Implementación:** Traducción de los modelos de diseño a código fuente ejecutable mediante la codificación y programación del software.
 - **Pruebas (Testing):** Verificación y validación de la funcionalidad y calidad del sistema.
 - **Despliegue:** Instalación y puesta en marcha operativa del sistema en un entorno de producción.
 - **Documentación y Capacitación:** Elaboración de manuales técnicos y guía para los usuarios finales.
-

2. Disciplinas de Gestión

Conjunto de actividades enfocadas en la planificación, monitoreo y control del proyecto de desarrollo de software.

Su propósito es organizar y coordinar personas y recursos para construir software de acuerdo con el presupuesto, alcance, plazos y la calidad acordada con el cliente.

Se dividen en dos grandes bloques de trabajo:

- **Planificación de Proyecto:** Es la actividad orientada a definir la "hoja de ruta" del proyecto, estableciendo qué se va a hacer, cuándo, cómo y quiénes serán los responsables, incluye:
 - **Definición del alcance y objetivos:** Determina los límites del proyecto y los resultados esperados
 - **Selección del proceso y ciclo de vida:** Adapta el modelo de desarrollo (secuencial, iterativo o ágil) al contexto del proyecto
 - **Estimación:** Pronóstica el esfuerzo, tiempo, costos y recursos necesarios

- **Gestión de riesgos:** Identifica, analiza y prepara planes de contingencia ante posibles eventos adversos
- **Programación y cronograma (Scheduling):** Establece el calendario de desarrollo y las relaciones de dependencia entre tareas
- **Definición de métricas y controles:** Determina qué indicadores y reuniones periódicas se utilizarán para medir el progreso
- **Monitoreo y Control de Proyectos:** Consiste en la supervisión activa del avance real en comparación con lo planificado, facilitando la toma de acciones correctivas.
 - Seguimiento de hitos
 - Control de cambios
 - Reportes de estado

Enfoque Tradicional: Se basa en una planificación inicial exhaustiva (*BRUF*), estimaciones cuantitativas, seguimiento de la ruta crítica y un control centralizado conducido por el Líder de Proyecto

Enfoque Ágil / Lean: La gestión se vuelve empírica, continua e iterativa; la planificación se distribuye en distintos horizontes (Release, Sprint, Daily) y el seguimiento se apoya en la transparencia del *backlog* y gráficos de avance (*Burnup/Burndown*)

3. Disciplinas de Soporte

Las disciplinas de soporte (también conocidas como *actividades sombrilla* o *paraguas*) son actividades transversales a los procesos de software que se aplican a lo largo de todo el ciclo de vida del proyecto.

Su objetivo es garantizar la integridad, trazabilidad y la calidad del producto a lo largo de su evolución:

Tenemos tres disciplinas de soporte principales:

- **Gestión de Configuración del Software (SCM):** Control de cambios, administración de versiones e ítems de configuración para mantener la integridad del producto.
- **Aseguramiento de Calidad (QA - Quality Assurance):** Evaluación y auditoría del cumplimiento de los estándares y procesos de calidad definidos.

- **Métricas:** Recolectan y evalúan datos cuantitativos del proceso, del proyecto y del producto. Transforma mediciones en indicadores para facilitar la toma de decisiones.
-

Ejemplos de grandes proyectos de software fallidos y exitosos.

Proyectos de Software Fallidos (Errores Informáticos Costosos):

- **Destrucción del Mariner I (1962):** La sonda espacial de la NASA con destino a Venus se desvió de su trayectoria y debió ser destruida a los 293 segundos del despegue debido a que una fórmula manuscrita fue programada de manera incorrecta.
- **Catástrofe del Hartford Coliseum (1978):** El techo de un coliseo se derrumbó por el peso de la nieve pocas horas después de un evento masivo debido a un cálculo incorrecto en el software CAD usado para el diseño estructural.
- **El Gusano de Morris (1988):** Creado por Robert Tappan Morris, fue el primer ataque masivo a gran escala en Internet que provocó fallas y costos de limpieza masivos.
- **Error de Cálculo de Intel (1994):** Un fallo en el procesador Pentium de Intel obligó a la sustitución masiva de los procesadores defectuosos.
- **Explosión del Cohete Ariane 5 (1996):** El cohete de la Agencia Espacial Europea explotó tras el despegue porque un número real de 64 bits relacionado con la velocidad se convirtió erróneamente a un entero de 16 bits
- **Mars Climate Orbiter (1999):** La NASA perdió contacto con la sonda al intentar orbitar Marte debido a que un programa calculaba la distancia en unidades inglesas (pulgadas, pies, libras) mientras que otro utilizaba el sistema métrico decimal
- **Fallas en Vehículos Toyota (2010 / Prius):** Retiro de más de 400.000 vehículos híbridos por un fallo en el software del sistema de frenos antibloqueo (ABS)
- **Mitigación del Bug del Año 2000 (Y2K):** Gran despliegue de recursos para corregir software antiguo que no estaba programado para interpretar el cambio de milenio

Proyectos de Software Exitosos y Validación de Producto:

- **Proyecto Apollo (NASA):** Destacado históricamente por el liderazgo de **Margaret Hamilton**, ingeniera principal de software que desarrolló el código de vuelo a bordo para las misiones espaciales de la NASA
- **Dropbox (Validación de Riesgo con MVP):** Para evitar el enorme riesgo técnico de construir una infraestructura distribuida compleja sin saber si existía demanda, el fundador creó un video demostrativo de 3 minutos como Producto Mínimo Viable (MVP) y disparó el número de clientes de la noche a la mañana.
- **Facebook:** En 2004, con ingresos casi nulos, validó su hipótesis de valor al demostrar que más del 50% de los usuarios registrados regresaban al sitio diariamente y que tres cuartas partes de los estudiantes de Harvard lo usaban en un mes
- **Productos con Alto Valor de Experiencia (MLP):** Casos como **Spotify, Instagram y WhatsApp** representan proyectos altamente exitosos que evolucionaron más allá de ser solo funcionales (MVP), logrando la lealtad masiva del usuario (*Minimum Lovable Product*)

Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.

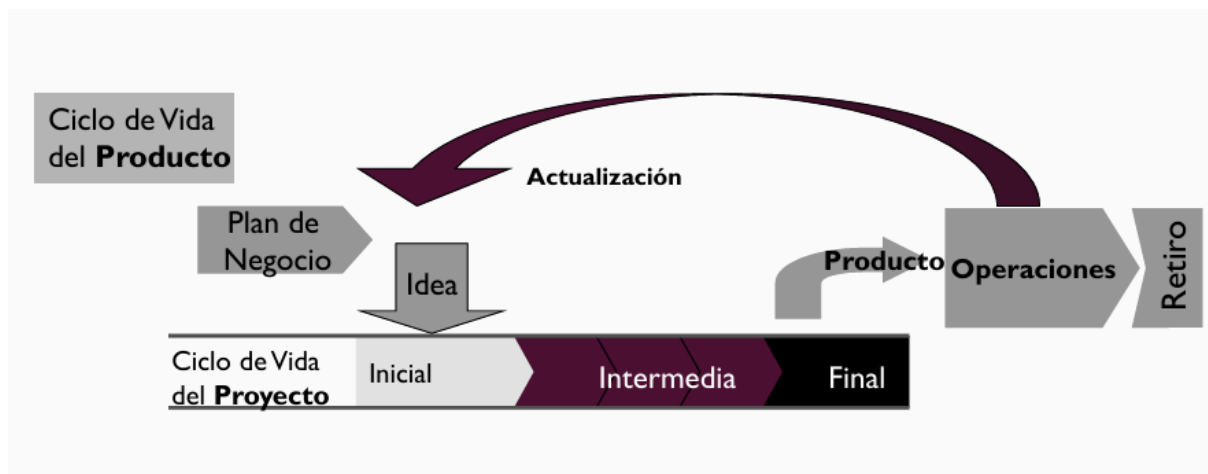
Un **ciclo de vida (o modelo de proceso)** es una representación simplificada y estructurada de un proceso de software, que grafica la serie de pasos a través de los cuales un producto o proyecto progresa a lo largo del tiempo. Define las actividades, el orden en el que se llevan a cabo y los criterios para avanzar entre fases.

Relación entre el Ciclo de Vida del Producto y del Proyecto

El **ciclo de vida del proyecto** comprende desde la planificación e inicio del desarrollo hasta la entrega e implementación del software.

El **ciclo de vida del producto** es más amplio: comienza con la idea o necesidad de negocio, incluye el desarrollo del proyecto, continúa con la operación, mantenimiento y evolución, y finaliza con el retiro del producto.

Por lo tanto, **el ciclo de vida del producto es más extenso que el del proyecto**, y un mismo producto puede involucrar **múltiples proyectos** durante su existencia.



Proceso de Desarrollo de Software: Es el conjunto completo de actividades, prácticas y herramientas que se utilizan para desarrollar y mantener software y sus productos asociados.

Clasificación de los Ciclos de Vida

- **Modelo Secuencial (Cascada puro, Modelo en V):**
 - Dispone las actividades en una secuencia lineal ordenada
 - Una fase no puede iniciar hasta que la anterior haya sido completada, revisada y aprobada
 - Es un enfoque formal dirigido por la documentación donde los requerimientos se congelan al inicio (*Big Requirements Up Front - BRUF*).
 - Es conveniente en escenarios de **baja incertidumbre**, donde los requerimientos son estables y completamente claros desde el inicio.
- **Modelo Iterativo (Marcos Ágiles, Entrega por etapas):**
 - El desarrollo se divide en iteraciones cortas donde cada una entrega un incremento de software funcional y potencialmente utilizable.
 - Integra en cada ciclo la especificación, el desarrollo y la validación.
 - Promueve la integración continua del cliente para obtener retroalimentación temprana y adaptar el sistema con un menor costo de retrabajo
 - Ideal para proyectos con **alta incertidumbre**, volatilidad de requerimientos y necesidad de entregas tempranas al cliente.
- **Modelo Recursivo (Modelo en Espiral):**

- Representa el proceso como una espiral continua dividida en cuadrantes donde cada vuelta constituye una fase del proyecto.
- Se enfoca en el análisis y la gestión explícita de riesgos en cada iteración antes de avanzar con la construcción.
- Utilizado en sistemas grandes, complejos y de alto riesgo donde no es factible congelar requerimientos sin evaluación previa.

Ventajas y desventajas de los ciclos de vida.

Modelo Secuencial:

- **Ventajas:**

- Permite realizar gran parte de la planificación al inicio y establecer un plan de trabajo definido.
- Los hitos y entregables requeridos al cierre de cada fase proporcionan indicadores claros de progreso
- Caminos lineales y rígidos que proporcionan una guía clara de trabajo, especial para equipos poco experimentados.
- Facilita el uso de contratos y mecanismos de gestión tradicionales

- **Desventajas:**

- Los requisitos se definen tempranamente y modificarlos más adelante puede resultar costoso y complejo.
- El cliente no obtiene una versión ejecutable hasta etapas avanzadas o al final del proyecto
- Descubrir un fallo de diseño o requerimiento al final puede generar costos mayores de corrección.
- Exige una elevada documentación del avance y tareas administrativas para el control del proyecto

Modelo Iterativo:

- **Ventajas:**

- Se adapta mejor a requerimientos cambiantes y emergentes
- El cliente obtiene software funcional desde etapas tempranas del proyecto
- Permite validar el producto con usuarios reales mediante entregas y demostraciones frecuentes.
- Permite evaluar aspectos técnicos y funcionalidades críticas antes de completar todo el desarrollo

- **Desventajas:**

- Resulta más complejo medir el progreso mediante documentos convencionales y métricas tradicionales

- Sin refactorización frecuente, añadir constantemente cambios puede degradar la arquitectura y aumentar la complejidad del código
- Choca con esquemas contractuales que requieren precios, plazos, entregables o un alcance fijo de antemano.
- Incorporar cambios puede ser difícil cuando el sistema debe mantener funcionalidades antiguas necesarias para la operación.
- Requiere un mayor involucramiento del cliente, cosa que no siempre es posible o deseada por el cliente

Modelo Recursivo:

- **Ventajas:**
 - Es el modelo adecuado para identificar, analizar y resolver riesgos técnicos antes de avanzar a etapas posteriores
 - Combina una planificación estructurada con iteraciones y prototipado evolutivo.
 - Ideal para proyectos complejos y de larga duración
 - Cubre todo el ciclo de vida del producto
- **Desventajas**
 - Elevado costo de gestión
 - Dificultad de aplicación en contratos fijos
 - Complejidad para establecer hitos

Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.

La elección del modelo de **ciclo de vida** adecuado no es un proceso dogmático ni existe una solución única aplicable a todos los proyectos. Para seleccionar el modelo que mejor responda a las necesidades de la organización y a las características del producto, se deben ponderar diversos **criterios de selección**:

1. Claridad de Requerimientos e Incertidumbre

- **Baja incertidumbre y requerimientos estables:** Cuando el problema y el negocio se comprenden con claridad desde el inicio y los requerimientos son exhaustivos, el modelo **Secuencial (Cascada o Modelo en V)** es el indicado, pues permite planificar y definir el alcance por adelantado.

- **Alta incertidumbre y requerimientos cambiantes:** En entornos dinámicos donde las necesidades evolucionan, la mejor opción es un ciclo **Iterativo e Incremental**, que tolera el cambio y ajusta el producto mediante retroalimentación constante.
 - **Elevados riesgos técnicos o funcionales:** Cuando existen grandes incertidumbres que podrían comprometer al proyecto, conviene aplicar un modelo **Recursivo**, enfocado en identificar, analizar y mitigar riesgos antes de continuar con la construcción.
-

2. Plazos y Necesidad de Entregas del Cliente

- **Urgencia de valor temprano (Time-to-Market):** Si el cliente necesita obtener versiones utilizables lo antes posible para captar mercado o validar hipótesis, se recomiendan ciclos **Iterativos e Incrementales** o estrategias de entrega por etapas.
 - **Entrega única y global:** Si por naturaleza del producto o restricciones del cliente no es factible realizar entregas parciales, se adaptan mejor los modelos **Secuenciales** o **Recursivos**.
-

3. Tamaño, Complejidad y Criticidad del Producto

- **Sistemas pequeños o de complejidad moderada:** Funcionan eficazmente con ciclos **Iterativos e Incrementales** informales y ágiles.
 - **Sistemas grandes, complejos o críticos:** Los productos de gran escala, con requerimientos estrictos de seguridad y fiabilidad, exigen una mayor arquitectura previa y control de riesgos, siendo adecuados los modelos **Recursivos (Espiral)** o **Secuenciales** con especificaciones rigurosas.
-

4. Involucramiento del Cliente y Contratación

- **Cliente disponible y colaborativo:** Potencia los ciclos **Iterativos e Incrementales**, ya que puede evaluar cada incremento y ajustar el rumbo.
 - **Cliente con baja disponibilidad o contratos de alcance/precio fijo:** Favorece modelos **Secuenciales** basados en planes rígidos y especificaciones formalmente aprobadas.
-

5. Experiencia del Equipo y Cultura Organizacional

- **Equipos poco experimentados:** Un ciclo **Secuencial** brinda una estructura rígida por etapas y documentos que ayudan a coordinar el trabajo.
- **Equipos maduros y autogestionados:** Pueden desempeñarse con alta eficiencia bajo modelos **Iterativos e Incrementales** respaldados por procesos empíricos.

6. Paradigma de Proceso Adoptado

- **Procesos Definidos (Deterministas):** Tienen la flexibilidad de **admitir cualquier ciclo de vida** (secuencial, iterativo o recursivo) según la estrategia del proyecto.
 - **Procesos Empíricos (Ágiles / Lean):** Admiten **únicamente ciclos de vida iterativos e incrementales**, descartando los modelos secuenciales por su rigidez frente al cambio.
-

Matriz Resumen de Selección

Ciclo de Vida	Paradigma que lo admite	Criterios Principales de Elección
Secuencial	Procesos Definidos	Alta certeza, requerimientos claros/estables, entrega única total obligatoria y necesidad de estructura rígida para la gestión.
Iterativo	Procesos Definidos y Empíricos	Requerimientos volátiles, alta incertidumbre, necesidad de entregas tempranas e involucramiento continuo del cliente.
Recursivo	Procesos Definidos	Proyectos grandes, complejos e innovadores donde el factor determinante es la gestión y mitigación constante de riesgos.

Influencia en la Administración de Proyectos

La selección del ciclo de vida condiciona directamente todas las actividades de gestión:

Gestión de la triple restricción (alcance, tiempo y costo)

- *Enfoque tradicional: el alcance se fija desde el inicio, y en función de él se estiman los plazos y los costos.*
- *Enfoque ágil/iterativo: el tiempo (duración de las iteraciones) y los recursos se mantienen fijos, mientras que el alcance es la variable que se ajusta y reprioriza constantemente según las necesidades del negocio.*

Medición del avance y métricas

- *En los modelos tradicionales, el avance se mide comparando lo ejecutado contra lo planificado y contra los documentos aprobados.*
- *En los modelos iterativos, el principal indicador de progreso es la entrega continua de software funcional, junto con la velocidad de trabajo del equipo.*

Tratamiento del cambio y del riesgo

La elección del ciclo de vida determina si el cambio se percibe como un riesgo costoso que requiere mecanismos formales de control (enfoque tradicional), o como un insumo valioso para mejorar el producto a través del aprendizaje continuo (enfoque iterativo).

Procesos de Desarrollo Empíricos vs Definidos

Proceso de Software: Conjunto de actividades, métodos, prácticas y transformaciones para desarrollar o mantener un sistema.

Las actividades del Proceso de Desarrollo son: la especificación de requerimientos, el análisis, el diseño, la implementación y la prueba. La relación y el orden de las actividades dependen del ciclo de vida seleccionado.

Los tres factores determinantes del Proceso de Software son:

- **Procedimientos y métodos:** Establecen *qué actividades se deben realizar y cuáles no*, garantizando la visibilidad, la transparencia y la repetibilidad del proceso.
- **Personas motivadas, capacitadas y con habilidades:** Es el factor más importante y determinante. Debido a que el desarrollo de software es una actividad humano-intensiva e intelectual, la capacidad, la motivación y la experiencia del equipo influyen directamente sobre la calidad final del producto
- **Herramientas y equipos:** comprenden el soporte material y tecnológico que permite transformar las entradas en salidas. Se recomienda automatizar todo lo posible, para que las personas se concentren en las tareas que realmente requieren su criterio.

¿Por qué no existe un "proceso ideal"?:

Los procesos de software son complejos, y la mayoría de las organizaciones diseñan el

propio en función de las capacidades del personal y las características de los sistemas que desarrollan. Por ejemplo: los sistemas críticos requieren procesos muy estructurados (proceso definido), mientras que los sistemas empresariales con requerimientos cambiantes se benefician de procesos más flexibles (filosofía ágil).

Consideraciones adicionales:

1. Las actividades del proceso varían según la organización y el sistema, pero siempre deben incluir productos, roles, responsabilidades y condiciones.
2. Un proceso debe estar explícitamente modelado para poder ser administrado.
3. Un mismo proceso de desarrollo puede aplicarse a distintos ciclos de vida, elegido según la estrategia del proyecto.
4. Las personas usan herramientas y equipos para llevar a cabo los procedimientos del proceso.

Procesos Definidos

Los Procesos Definidos poseen un enfoque de desarrollo determinista, inspirado en las líneas de producción.

- Asumen que el proceso se puede planificar de antemano y que, ante las mismas entradas, se obtienen siempre las mismas salidas, con los mismos resultados.
- Se basa en la capacidad de formalizar y estandarizar procedimientos para lograr repetibilidad de las tareas y previsibilidad en términos de costos, tiempos, esfuerzos y riesgos
- La documentación y el seguimiento cobran mayor importancia
- Tiene la flexibilidad de poder utilizar cualquier modelo de ciclo de vida
- En la práctica, no es posible garantizar los mismos resultados ante entradas similares ya que el proceso de desarrollo de software es dependiente del contexto, la complejidad y el factor humano.

Procesos Empíricos (Ágiles / Lean)

Los procesos empíricos se basan en la **experiencia** de las personas, la observación directa y en la evidencia acumulada dentro de un contexto particular. A diferencia de los procesos definidos el enfoque empírico reconoce que el desarrollo de software es una actividad de alto contenido intelectual donde resulta imposible predecir o congelar todas las variables por adelantado

- Reconocen que se trabaja en entornos complejos, con numerosas variables, por lo que repetir un proceso no garantiza obtener los mismos resultados
- La administración del proyecto no proviene del seguimiento de un plan prefijado, sino de evaluar continuamente el estado real del producto y del proceso
- El conocimiento se genera en un contexto específico de cada equipo y proyecto y no se puede extrapolar sin adaptación.
- **Patrón de aprendizaje:** *Asumir → Construir → Retroalimentar → Revisar → Adaptar*
- **Sólo admiten ciclos de vida iterativos e incrementales.** El modelo secuencial tradicional queda descartado debido a su rigidez ante los cambios.

Los Tres Pilares del Empirismo

1. **Transparencia:** El proceso, los aspectos del trabajo y estado real del producto deben ser visibles para todos aquellos que reciben y ejecutan el trabajo. Permite convertir el conocimiento implícito de cada individuo en conocimiento explícito del equipo.
2. **Inspección:** Evaluación regular del producto, los procesos y el avance con el fin de detectar desvíos no deseados.
3. **Adaptación:** Capacidad de realizar un ajuste inmediato si se detecta un desvío en algún aspecto del proceso o si el producto no cumple con la calidad requerida.

Comparativa de Procesos: Definidos vs. Empíricos

Característica	Procesos Definidos 	Procesos Empíricos 
Origen / Analogía	Línea de ensamblaje industrial	"Caminitos" o sendas hechas al andar
Premisa Principal	Determinista: entradas iguales → salidas iguales	No determinista: adaptado al contexto y la experiencia
Estrategia de Control	Planificación rígida y predictibilidad	Inspección y adaptación continuas
Ambiente Ideal	Problemas simples o con baja incertidumbre	Dominios complejos, creativos y cambiantes
Ciclos de Vida Admitidos	Secuencial, Iterativo o Recursivo	Únicamente Iterativo e Incremental
Tratamiento del Cambio	Desviación costosa que se busca evitar o controlar	Insumo valioso para generar aprendizaje y valor

Componentes de un Proyecto

Un proyecto es un esfuerzo temporal y estructurado que integra personas y recursos para desarrollar un producto o servicio, obteniendo un resultado único.

Su objetivo es organizar y gestionar el trabajo necesario para desarrollar el sistema, controlando su complejidad.

Componentes Estructurales

El proyecto actúa como núcleo organizativo que vincula cuatro elementos fundamentales:

- **Personas:** Es el componente principal, ya que el desarrollo de software es una actividad *humano-intensiva*. Las personas se incorporan al proyecto para aportar sus conocimientos, habilidades y realizar el esfuerzo intelectual
- **Proceso:** Conjunto estructurado de actividades y prácticas que se adoptan y adaptan según las características del proyecto.
- **Herramientas:** Soporte tecnológico que facilita o automatiza las actividades mecánicas del proceso.
- **Producto:** Resultado obtenido como consecuencia de la ejecución del proyecto.

La Triple Restricción y la Calidad

La administración del proyecto requiere balancear de forma continua tres variables operativas para garantizar la Calidad. Esta responsabilidad recae en el Líder del Proyecto

- **Alcance:** Representa todo el trabajo y la funcionalidad requerida que debe incluirse para entregar el producto
- **Tiempo (Cronograma):** Duración estimada, plazos y fechas comprometidas para la entrega
- **Costo (Presupuesto):** Esfuerzo del personal y recursos económicos asignados

Principio de Equilibrio: No es posible fijar las tres variables a la vez; si dos se mantienen fijas, la tercera debe ajustarse para no degradar la calidad del software

- **Enfoque Tradicional (alcance fijo):** Se define y fija el alcance desde el inicio, estimando a partir de él el tiempo y el costo
- **Enfoque Ágil (alcance variable):** Se fijan el tiempo (sprints) y los recursos/costos, manteniendo el alcance variable y adaptable

Artefactos de Administración del Proyecto

¿Qué es la Administración de Proyectos?

Es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos del mismo.

Administrar un proyecto incluye:

- Identificar requerimientos
- Establecer objetivos claros y alcanzables
- Adaptar las especificaciones, planes y el enfoque a los intereses de los involucrados (stakeholders)

El proyecto se gestiona a través de dos actividades principales: **Planificación** y **Monitoreo/Control**, apoyándose en los siguientes artefactos:

- **Plan de Proyecto (Plan de Desarrollo de Software):** Documento formal que consolida los objetivos, cronogramas, estimación de esfuerzos, asignación de roles, gestión de riesgos y planes de soporte.
- **Ítems de Configuración de Proyecto:** Artefactos de soporte cuyo ciclo de vida está acotado a la duración del proyecto o de una iteración (cronogramas, planes de pruebas, matriz de riesgos, reportes de defectos).

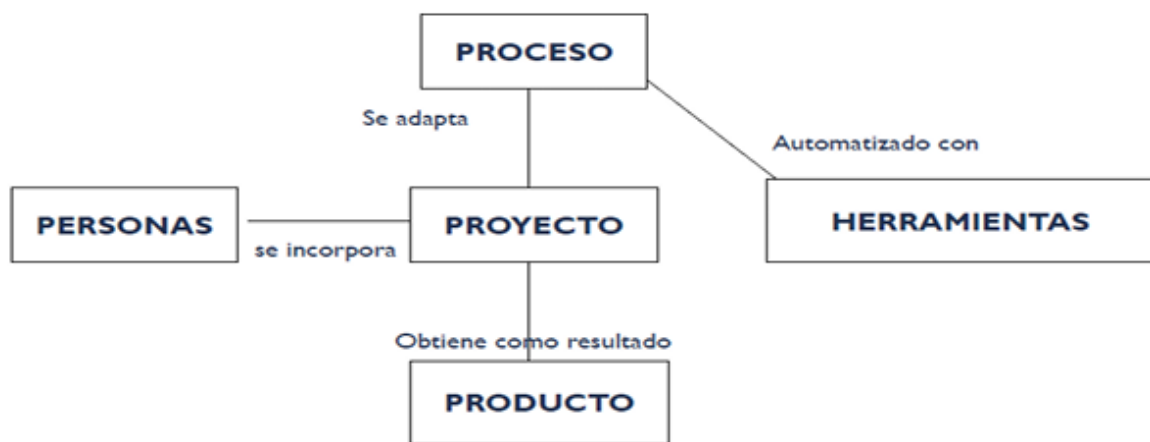
Características de un Proyecto

Todo proyecto se distingue por tres propiedades fundamentales:

1. **Resultado Único e Irrepetible:** Aunque existan proyectos similares, el resultado final y las circunstancias (calendario, presupuesto, entorno) son siempre singulares.
2. **Orientado a objetivos:** Los objetivos guían el proyecto. Estos no pueden ser ambiguos sino que deben ser definidos de forma clara, además tienen que ser factibles, alcanzables y medibles.
3. **Duración Limitada:** Tiene un inicio y fin definidos, liberando los recursos al ser completado.

Vinculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.

El axioma central establece que el **Proceso**, soportado y automatizado con **Herramientas**, se **instancia y adapta** al **Proyecto**, al cual se incorporan **Personas**, obteniendo como resultado final un **Producto**.



Proceso (El marco de trabajo o "Cómo"): Es el conjunto estructurado de actividades, métodos, prácticas y transformaciones que guían el desarrollo y mantenimiento del software.

Proyecto (El vehículo organizativo o "Dónde"): Es la unidad o esfuerzo temporal donde se aplica el proceso de forma concreta. En el proyecto se incorporan y coordinan las **personas** y las **herramientas tecnológicas**, administrando recursos, plazos y costos para manejar la complejidad.

Producto (El resultado entregable o "Qué"): Es el conjunto de programas ejecutables, documentación asociada, modelos y datos obtenidos como consecuencia de ejecutar las actividades del proceso dentro del proyecto para satisfacer las necesidades del cliente.

Gestión de Producto

La **Gestión de Productos de Software** (*Product Management*) es la disciplina encargada de guiar la estrategia, desarrollo y evolución de un producto informático para asegurar que entregue valor real a los usuarios y al negocio.

El Ecosistema de los "Mínimos"

Para gestionar la incertidumbre y trazar el **Roadmap** de liberación del producto, se definen distintas etapas de maduración:

UVP (Propuesta de Valor Única): Expresa qué beneficio diferencial ofrece el producto al cliente y por qué debería elegirlo frente a otras alternativas.

MVP (Producto Mínimo Viable): Versión del producto que permite a un equipo obtener *aprendizaje validado* sobre los clientes con el mínimo esfuerzo. Sirve para validar hipótesis con usuarios reales.

Pivote (Pivot): Ocurre cuando la retroalimentación del MVP demuestra que los usuarios apuntan hacia una necesidad distinta a la contemplada originalmente

MMP/MMR (Minimum Marketable Product/Release): Primera versión comercial suficientemente completa para salir al mercado y generar valor al cliente.

MMF (Minimum Marketable Feature) y MVF (Minimum Viable Feature):

- **MMF:** funcionalidad mínima que puede entregarse y aportar valor comercial al producto, generalmente en áreas donde existe alta certeza sobre lo que necesitan los usuarios.
- **MVF:** funcionalidad mínima utilizada para validar una hipótesis en un área de alta incertidumbre dentro de un producto existente. Puede considerarse como un mini-MVP aplicado a una funcionalidad.

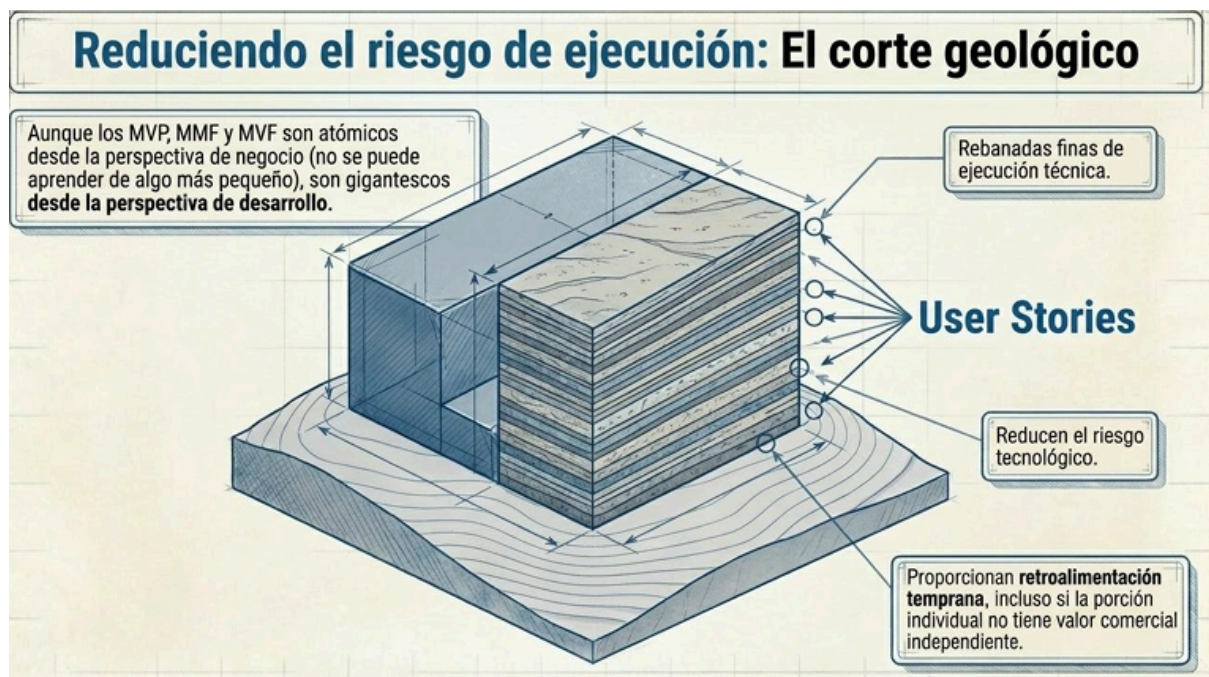
MLP (Minimum Lovable Product / Producto Mínimo Adorable): Producto mínimo que además de la viabilidad busca generar una experiencia atractiva y conexión con el usuario (indispensable en mercados altamente competitivos)



MVP (Producto Mínimo Viable)	MMP (Producto Mínimo Comercializable)	MLP (Producto Mínimo Adorable)
Enfoque: Entender necesidades Validar conceptos rápido y barato.	Enfoque: Amplio alcance / Atender a grandes bases de usuarios.	Enfoque: Diseñar para el deleite / Conexión emocional.
Foco Clave: Investigación, Código base, Pruebas de usuario.	Foco Clave: Estabilidad técnica, Lanzamiento al mercado.	Foco Clave: UI avanzada, Impacto emocional, Reducción de fricción.
Resultado: Informar el alcance del producto.	Resultado: Atracción de mercado masivo.	Resultado: Lealtad temprana y retención.

Ejecución Técnica: User Stories y "Dinosaurio Carpaccio"

Aunque los MVP, MMF y MVF son atómicos desde la perspectiva de negocio, para el desarrollo técnico representan grandes bloques. Para reducir el riesgo, se aplica la técnica del **corte geológico** o "dinosaurio carpaccio": rebanar las características en porciones verticales finas (**User Stories**) que atraviesan la interfaz, la lógica de negocio y la base de datos, garantizando retroalimentación constante.



Dinosaurio Carpaccio 🦖

La idea es **dividir una funcionalidad grande en pequeñas porciones verticales**.

Cada porción atraviesa todas las capas necesarias:

Interfaz → lógica de negocio → base de datos

En lugar de hacer primero toda la interfaz, después toda la lógica y finalmente la BD, se construye una pequeña funcionalidad **completa de punta a punta**.

Esto permite:

- Entregar valor rápidamente
- Obtener feedback temprano
- Reducir riesgos
- Detectar problemas técnicos de forma temprana.

¿Qué es la Filosofía Ágil?

- La agilidad **no es un proceso ni una metodología**, sino una ideología guiada por un conjunto de valores y principios para encarar proyectos complejos.
- **Fundamento Empírico:** Se apoya en el **control de procesos empíricos**, sustentado en tres pilares fundamentales: Transparencia, Inspección y Adaptación.
- **Respuesta a la rigidez tradicional:** Surge para mitigar los problemas del desarrollo tradicional orientado al plan inicial exhaustivo (*Big Requirements Up Front* - BRUF) y reducir el desperdicio.

"Hacer Ágil" vs. "Ser Ágil":

- **Hacer Ágil (Doing Agile):** Consiste en aplicar mecánicamente eventos, herramientas o ceremonias (tableros, reuniones diarias, sprints).
- **Ser Ágil (Being Agile):** Implica adoptar una cultura organizacional y mentalidad basada en la colaboración, la autonomía de equipos autoorganizados y la mejora continua.

Los 4 Valores Ágiles

El manifiesto aclara que, aunque existe valor en los elementos de la derecha, **se valoran más los elementos de la izquierda:**

1. **Individuos e interacciones** por sobre *procesos y herramientas*: Al ser el desarrollo una actividad humano-intensiva, la comunicación fluida entre personas motivadas impacta más que la tecnología utilizada.

2. **Software funcionando** por sobre *documentación exhaustiva*: La medida real de avance es entregar código ejecutable y valioso, evitando la burocracia documental innecesaria.
3. **Colaboración con el cliente** por sobre *negociación contractual*: El cliente y los interesados se integran continuamente al equipo para definir el producto, en lugar de discutir contratos o cláusulas rígidas.
4. **Respuesta ante el cambio** por sobre *seguir un plan*: Se abraza la variabilidad del entorno y se promueve la adaptación en lugar de cumplir ciegamente un plan rígido desde el inicio.



Estos valores se materializan a través del manifiesto ágil:



¿Qué es desperdicio? Todo aquello que no agregue valor directo al cliente o al producto final

¿Entonces qué significa ágil? Es el balance justo entre no tener ningún proceso y tener demasiados. Exige una menor cantidad de documentación, es adaptable en lugar de predictivo y orientado a las personas en lugar de los procesos.

¿Por qué ir a Agile? Se adopta Agile porque permite desarrollar software en entornos complejos y cambiantes, adaptándose a nuevas necesidades, eliminando el desperdicio, reduciendo el riesgo y entregando valor de forma frecuente.

Requerimientos Ágiles

En la Ingeniería de Software, los **requerimientos ágiles** representan un cambio radical de paradigma respecto a los enfoques tradicionales. En lugar de intentar definir exhaustivamente todos los requerimientos al inicio del proyecto (evitando el BRUF), la agilidad abraza el cambio y propone gestionar los requerimientos de forma iterativa, incremental y *Just in Time*.

"Un error común es creer que 'ágil' significa ausencia de documentación. En realidad, lo que propone es reemplazar la documentación exhaustiva y anticipada (BRUF) por una documentación progresiva y oportuna (Just in Time)"

Fundamentos de los Requerimientos Ágiles

Los requerimientos en entornos ágiles se sostienen sobre tres pilares principales:

- **Foco en el valor:** El objetivo es entregar software funcional que resuelva problemas reales de negocio.
- **"Sólo lo suficiente":** Releva únicamente el nivel de detalle necesario para avanzar
- **Trabajo colaborativo:** Fomentar la comunicación directa (cara a cara) entre personas del negocio (Product Owner) y técnicos durante todo el proyecto

La Triple Restricción Ágil

A diferencia del enfoque tradicional donde el alcance es fijo y se estiman el tiempo y los costos, el enfoque ágil invierte la pirámide: **el tiempo (sprints) y los recursos son fijos**, mientras que **el alcance es variable** y se reordena constantemente según el valor que aporta al cliente. Además, se reemplaza la Especificación de Requerimientos de Software (ERS) por el **Product Backlog** como un contenedor dinámico y transitorio de requerimientos.

Niveles de Requerimientos y Abstracción

Los requerimientos se dividen según el dominio:

- **Dominio del Problema (Interesa al Cliente):**
 - **Requerimientos de Negocio:** Definen los objetivos de alto nivel de la organización.
 - **Requerimientos de Usuario:** Expresan lo que los usuarios necesitan hacer para lograr ese valor de negocio.
 - **Dominio de la Solución (Interesa al Equipo Técnico):**
 - **Requerimientos de Software:** Incluyen los requerimientos funcionales, no funcionales y de implementación que detallan cómo el sistema construirá la solución.
-

Artefactos y Conceptos Complementarios

Definition of Ready (DoR): Acuerdo entre el Product Owner y el equipo de desarrollo sobre las condiciones de madurez que debería cumplir una Historia de Usuario o ítem del Product Backlog para ser seleccionada en el Sprint Planning y entrar a un Sprint.

Suele incluir:

- Criterios de aceptación claros y definidos
- Cumplimiento del modelo INVEST

Si bien la DoR no es un artefacto oficial de Scrum, es una práctica ampliamente utilizada por equipos ágiles.

- Evita que una US mal entendida o demasiado grande entre al Sprint y genere bloqueos
- Facilita el proceso de estimación y el Sprint Planning
- Se revisa y ajusta durante el Refinamiento del Product Backlog

Definition of Done (DoD): Descripción formal del estado del Incremento cuando cumple con los estándares de calidad requeridos para el producto.

- Un ítem que no cumple el DoD no puede formar parte del Incremento ni presentarse en el Sprint Review. En su lugar, regresa al *Product Backlog* para ser considerado en iteraciones futuras
- Aunque todo el Scrum Team participa en **crearlo**, son específicamente **los Developers quienes deben cumplirlo**
- Cumplir el DoD significa que el trabajo está potencialmente liberable
- El estándar de la DoD **nunca se debilita ni se flexibiliza ante la presión del tiempo o plazos de entrega**; si se necesita ajustar el trabajo durante el Sprint, se reduce el alcance, pero jamás la calidad
- Los *Developers* son los encargados de adherirse y aplicar la DoD en cada Sprint
- Cuando varios equipos trabajan coordinadamente en un mismo producto, todos los equipos deben definir y compartir obligatoriamente la misma DoD
- El DoD es el estándar de calidad del **Incremento** (el resultado acumulado y entregable del Sprint).
- Una Historia de Usuario individual **no "tiene" su propio DoD** — lo que ocurre es que, cuando una o más historias cumplen esos criterios de calidad, en conjunto **dan origen** a un Incremento.

Spikes: Tipo especial de Historia de Usuario orientada a reducir riesgos y el exceso de incertidumbre sobre un requerimiento. Surge cuando la incertidumbre es tal que impide que una US pueda ser estimada de forma fiable, evitando que cumpla con la DoR e ingrese al Sprint.

- **Spike técnica:** se utiliza para evaluar e investigar enfoques técnicos y tecnológicos.
- **Spike funcional:** se utiliza cuando existe incertidumbre sobre cómo el usuario va a interactuar con el sistema para satisfacer sus necesidades.

BRUF (Big Requirements Up Front): Término utilizado para describir el enfoque tradicional de intentar relevar y documentar exhaustivamente todos los requerimientos del sistema al inicio del proyecto.

Principios relacionados a los Requerimientos Ágiles

Principios Ágiles

- 1- La prioridad es satisfacer al cliente a través de releases tempranos y frecuentes (2 semanas a un mes)
- 2 -Recibir cambios de requerimientos, aun en etapas finales
- 4 - Técnicos y no técnicos trabajando juntos TODO el proyecto
- 6 - El medio de comunicación por excelencia es cara a cara
- 11 - Las mejores arquitecturas, diseños y requerimientos emergen de equipos autoorganizados

Principios Lean

- 1- Eliminar el desperdicio
- 4 - Diferir Compromisos
- 6 - Ver el todo

User Stories

Las **Historias de Usuario** (*User Stories* - US) son una técnica para capturar y estructurar requerimientos de usuario en entornos ágiles.

Consisten en una descripción corta de una funcionalidad valuada desde la perspectiva de un usuario del sistema, expresada en el lenguaje del negocio.

A diferencia de las especificaciones tradicionales (como las ERS o Casos de Uso), las historias de usuario no son contratos rígidos ni descripciones exhaustivas; sino que expresan una intención y funcionan como **recordatorios/tokens para una conversación futura**.

Una US es una promesa de conversación.

Suelen ser redactadas por el **Product Owner**.

Anatomía de una Historia de Usuario (Las 3 "C")

1. **Tarjeta (Card)**: Es la descripción corta que sirve como recordatorio e ítem de planificación. Sigue una estructura estándar:

Como <rol>, yo puedo <actividad> de forma tal que <valor de negocio>

- **Rol:** *Quién realiza la acción o recibe el beneficio.*
 - **Actividad:** *La acción requerida que ejecutará el sistema.*
 - **Valor de negocio:** *La justificación del porqué se necesita la funcionalidad (permite priorizar).*
 - *Frase verbal / Título: Referencia corta no obligatoria para identificar la historia.*
2. **Conversación (Conversation):** Es la parte más importante. Es el diálogo continuo entre el Product Owner, el equipo de desarrollo y otros interesados relevantes para clarificar, refinar y acordar los detalles operativos de la historia antes de implementarla.
 3. **Confirmación (Confirmation):** Definición de los criterios y pruebas de aceptación que sirven para verificar y validar que la historia fue implementada correctamente.

Buscar Destino por Dirección

Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.

Criterios y Pruebas de Aceptación

- **Criterios de Aceptación:** Definen los límites de la historia y los requerimientos funcionales mínimos. Ayuda a que el equipo tenga una visión compartida de la US y derivar pruebas.
 - Definen una intención, no una solución técnica
 - Independientes de la implementación
 - Relativamente de alto nivel
- **Pruebas de Aceptación:** Sirven para capturar los escenarios detallados (exitosos y de fallo) derivados de las conversaciones.

Buscar Destino por Dirección Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme. Criterios de Aceptación: • La altura de la calle es un número. • La búsqueda no puede demorar más de 30 segundos.	Pruebas de Usuario ❑ Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura existente (pasa). ❑ Probar buscar un destino en un país y ciudad existentes, de una calle inexistente (falla). ❑ Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura inexistente (falla). ❑ Probar buscar un destino en un país inexistente (falla). ❑ Probar buscar un destino en País existente, ciudad inexistente (falla). ❑ Probar buscar un destino en un país y ciudad existentes, de una calle existente y demora más de 30 segundos (falla).
---	---

Modelo de Calidad INVEST

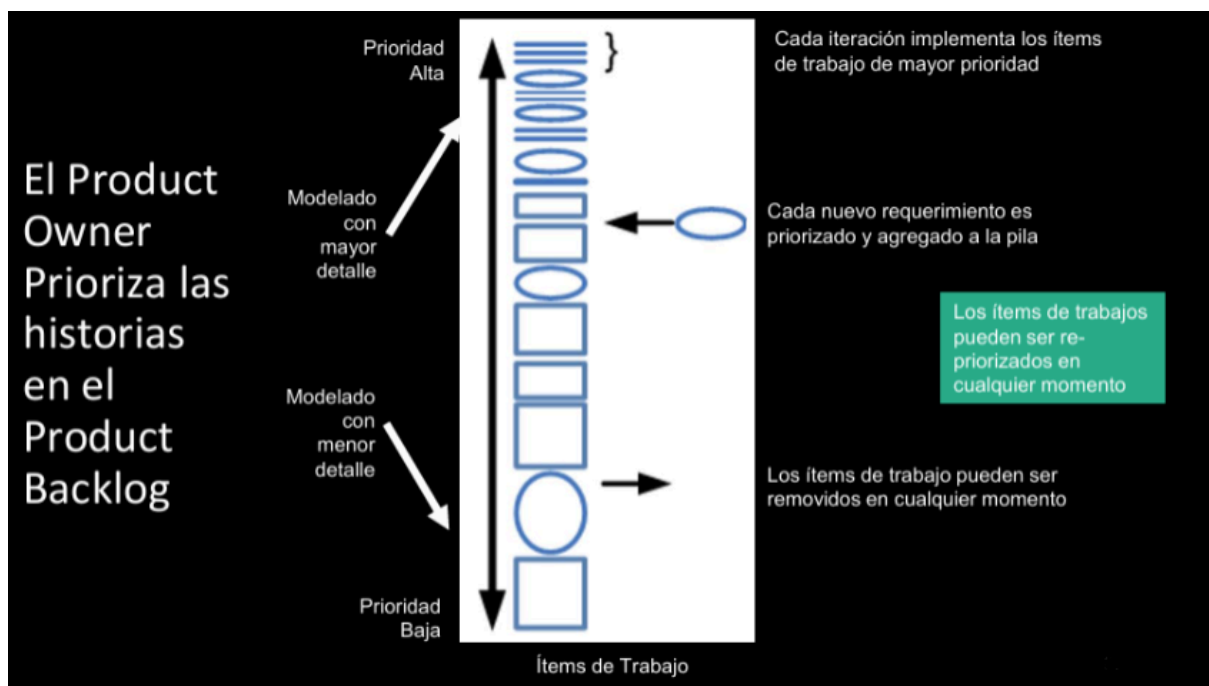
Para que una historia de usuario cumpla con la **Definición de Listo** (*Definition of Ready* - DoR) y pueda ingresar a una iteración o Sprint, debe cumplir con las características del modelo **INVEST**:

- **I - Independent (Independiente):** Se puede calendarizar e implementar en cualquier orden, sin dependencias rígidas con otras historias.
- **N - Negotiable (Negociable):** Describe el "qué" y no el "cómo", dejando espacio para negociar los detalles con el equipo.
- **V - Valuable (Valiosa):** Aporta un valor de negocio directo para el cliente o usuario.
- **E - Estimatable (Estimable):** Tiene la claridad suficiente para evaluar su tamaño, complejidad y esfuerzo (en *Story Points*).
- **S - Small (Pequeña):** Posee el tamaño adecuado para ser completada dentro de una iteración.
- **T - Testable (Testable):** Permite realizar pruebas concretas para demostrar su funcionamiento.

Niveles de Abstracción y Porciones Verticales

- **Temas:** Conjuntos de historias de usuario agrupadas por estar conceptualmente relacionadas.

- **Épicas:** Historias de gran tamaño que no caben en una sola iteración o no satisfacen el modelo INVEST. Se descomponen en historias más pequeñas para su desarrollo.
- **Porciones Verticales (Vertical Slices):** Las historias de usuario deben rebanar la arquitectura de forma vertical, abarcando la interfaz gráfica (GUI), la lógica de negocio y la persistencia/base de datos para entregar un incremento funcional completo.



Estimaciones Ágiles

Las **estimaciones ágiles** representan una aproximación numérica para predecir el tamaño del trabajo, determinar la factibilidad del proyecto y gestionar los riesgos en entornos con alcance variable e incertidumbre.

A diferencia de los enfoques tradicionales, la filosofía ágil sostiene que **el mayor beneficio de la estimación no es el número obtenido, sino el proceso de hacerla** (el debate, la alineación y el entendimiento compartido que logra el equipo).

- **Estimación Relativa vs. Absoluta:** Las personas no saben estimar con precisión en términos absolutos (horas o días exactos), pero son muy eficientes comparando. Estimar por comparación es más rápido y genera una mejor dinámica grupal.
- **¿Quién estima?:** La estimación la realiza **el equipo de desarrollo de forma colectiva** (quienes van a realizar el trabajo), evitando el sesgo de estimaciones hechas por líderes aislados. El **Product Owner** participa aclarando dudas sobre el negocio, pero **no estima**
- **"La precisión es cara" (Filosofía Lean):** Dedicar un esfuerzo excesivo a buscar estimaciones perfectas se considera un desperdicio en Lean, ya que la precisión absoluta en software no existe. Las estimaciones son *time-boxed* (con un tiempo límite asignado).
- **No son compromisos:** Las estimaciones son mediciones probabilísticas; no deben confundirse con promesas, compromisos o fechas límite fijas

¿Qué se estima? El Tamaño y los Story Points (SP)

En ambientes ágiles se estima **únicamente el tamaño relativo** de las User Stories. Para esto se utiliza el **Story Point (Punto de Historia)**, una unidad de medida abstracta que refleja el "peso" de una historia.

El Story Point **no es una medida basada en tiempo**, sino que integra **tres dimensiones clave**:

1. **Complejidad:** La cantidad de partes, reglas de negocio y relaciones de la historia.
2. **Esfuerzo:** El volumen de trabajo lineal necesario para implementarla.
3. **Incertidumbre / Riesgo:** La falta de información o desconocimiento del dominio del negocio o de la tecnología.

Escalas de Estimación

Dado que la complejidad del software tiende a incrementarse de manera exponencial, se utilizan escalas discontinuas:

- **Serie de Fibonacci:** 1, 2, 3, 5, 8, 13, 21, 34, 55, 89... (es la más común).
- **Talles de remeras:** S, M, L, XL, XXL (usado frecuentemente para elementos de alto nivel en el Product Backlog).

- **Regla de oro:** Una vez elegida la escala para un equipo, **no se cambia**, ya que alteraría el patrón de medición.

Decodificación de Valores (Fibonacci)

- **1 a 3 SP:** Funcionalidad pequeña a mediana (lo ideal para un sprint).
 - **5 SP:** Funcionalidad de tamaño medio.
 - **8 SP:** Funcionalidad grande; el equipo debe evaluar si es posible subdividirla.
 - **13 SP o más:** Se considera una **Épica** o historia demasiado grande; **debe dividirse obligatoriamente** en historias más pequeñas antes de poder ingresar a un Sprint.
-

Técnica de Estimación: Planning Poker

Es una técnica de consenso que combina juicio experto, analogía y desagregación.

Procedimiento de Trabajo

1. **Definir la historia canónica (Base Story):** El equipo selecciona una historia sencilla (de baja incertidumbre) a la que le asigna un valor base (ej. 1 SP) para usarla como punto de comparación o patrón de referencia.
 2. **Presentación y Discusión:** El Product Owner lee la User Story y el equipo le formula preguntas para aclarar dudas.
 3. **Votación Oculta:** Cada desarrollador selecciona en silencio una carta de su mazo con la estimación correspondiente.
 4. **Revelación Simultánea:** Todos muestran sus cartas al mismo tiempo para evitar el sesgo o la influencia entre compañeros.
 5. **Debate y Consenso:** Si hay consenso, se asigna el puntaje. Si existen discrepancias, los involucrados argumentan sus puntos de vista, se debate y se vuelve a votar hasta lograr la convergencia.
-

Métricas para la Planificación

A partir de las estimaciones de tamaño se derivan las métricas para planificar el proyecto:

- **Velocidad (Velocity):** Es una medición real (no una estimación) que calcula la suma de Story Points **100% completadas (DoD) y aceptadas por el Product Owner** al final de un Sprint. No se otorgan puntos parciales por historias "a medias".
- **Capacidad (Capacity):** Representa las horas de trabajo reales disponibles del equipo para un sprint (**Horas/día x Días laborables disponibles**).
- **Cálculo de la Duración del Proyecto:**

- $$\text{Duración Sprints} = \frac{\text{Total de Story Points del Product Backlog}}{\text{Velocidad Promedio del Equipo}}$$

Framework Scrum

Scrum es un marco de trabajo (*framework*) liviano diseñado para ayudar a generar valor a través de soluciones adaptativas a problemas complejos.

Se fundamenta en el **control de procesos empíricos** (donde el conocimiento proviene de la experiencia y la observación de hechos reales) y en el **pensamiento Lean** (centrado en reducir desperdicios y enfocarse en lo esencial). Utiliza un enfoque **iterativo e incremental** para optimizar la previsibilidad y controlar los riesgos del proyecto.

1. Los 3 Pilares del Empirismo

Scrum se sostiene sobre tres pilares fundamentales que permiten el control empírico del desarrollo:

- **Transparencia:** El proceso y el trabajo emergente deben ser visibles tanto para quienes realizan el trabajo como para quienes lo reciben. Los artefactos y decisiones deben basarse en una realidad percibida de manera clara por todo el equipo.
- **Inspección:** Evaluación frecuente y diligente de los artefactos y del avance hacia los objetivos definidos, permitiendo detectar desvíos o problemas no deseados.
- **Adaptación:** Ajuste inmediato del proceso o del producto en cuanto la inspección revela que algún aspecto se desvía de los límites aceptables.

Regla clave: la inspección sin adaptación se considera inútil o desperdicio.

2. Los 5 Valores de Scrum

- **Compromiso:** Lograr los objetivos y apoyarse mutuamente.
- **Enfoque:** Orientar el trabajo del Sprint hacia los objetivos establecidos.
- **Respeto:** Valorarse como personas capaces e independientes.

- **Apertura:** Transparencia sobre el trabajo y los desafíos encontrados.
 - **Coraje:** Afrontar problemas complejos y tomar las decisiones correctas.
-

3. El Scrum Team (Roles y Responsabilidades)

La unidad fundamental es el **Scrum Team** (generalmente de 10 o menos personas), **autogestionado** (decide internamente quién, cómo y cuándo hace el trabajo) y **multifuncional** (posee las habilidades necesarias para crear valor en cada iteración). No existen jerarquías internas ni sub-equipos.

Se componen de tres responsabilidades (*accountabilities*) principales:

- **Product Owner (PO):** Es el responsable de maximizar el valor del producto y de gestionar eficazmente el *Product Backlog* (crear, comunicar y priorizar los ítems).
- **Scrum Master (SM):** Es un líder que sirve al equipo y a la organización. Su responsabilidad es la efectividad del Scrum Team, capacitar en autogestión, guiar la adopción de Scrum y eliminar los impedimentos del entorno.
- **Product Developers:** Son quienes se comprometen a crear un Incremento útil y funcional en cada Sprint. Son responsables de crear el *Sprint Backlog*, planificar tácticamente el trabajo diario y garantizar la calidad adherida a la DoD (*Definition of Done*). *(En la evolución moderna del marco (H + IA), se contempla la integración de la IA como desarrollador automatizado, siempre bajo el principio "Human in the Loop" donde la responsabilidad final es humana).*

El Ecosistema Extendido (Roles de Entorno)

El equipo no opera de forma aislada; requiere de roles externos para sostener la autogestión y validar el valor comercial:

- **Stakeholders (Interesados):** Incluye personas, entidades o sistemas (clientes, usuarios finales, reguladores, entidades legales). Su función es proveer necesidades, retroalimentar al equipo y evaluar si el incremento genera un impacto real de negocio (*Outcome*).
 - **Supporter:** Es una coalición de líderes, gerentes y directivos que respaldan activamente la transformación ágil. Su rol es eliminar barreras burocráticas y organizacionales, brindando cobertura política al equipo para proteger el clima de autogestión.
-

Artefactos y sus Compromisos

En **Scrum**, los **artefactos** representan trabajo o valor y permiten maximizar la **transparencia** de la información. Cada artefacto incluye un compromiso formal asociado que sirve para medir el progreso.

Los 3 Artefactos Oficiales de Scrum y sus Compromisos

Product Backlog

Es una lista emergente, dinámica y ordenada de todo lo necesario para mejorar el producto. Constituye la única fuente de trabajo del equipo Scrum

Responsable: El **Product Owner** es quien tiene la palabra final sobre el contenido y el orden del backlog — es responsable de desarrollar y comunicar el **Product Goal**, y de ordenar los ítems. Sin embargo, puede recibir aportes del equipo y los stakeholders, y delega a los **Developers** la decisión sobre el **tamaño/estimación** de cada ítem.

Compromiso asociado → Product Goal:

- Describe un **estado futuro del producto**; funciona como la meta estratégica de largo plazo.
- Es, literalmente, **parte del propio Product Backlog** — no es un documento externo o separado.
- El equipo debe **cumplir o abandonar** un Product Goal antes de asumir el siguiente.

A menudo es descrito como un “organismo vivo” (en contraposición a un plan estático tradicional como la ERS), un concepto que conecta directamente con Requerimientos Ágiles, donde el Backlog reemplaza a la ERS como un contenedor transitorio y dinámico de requerimientos.

Sprint Backlog

Plan táctico creado por y para los desarrolladores con el fin de alcanzar la consecución del Sprint Goal

Composición del Sprint Backlog:

Reúne tres elementos definidos durante el Sprint Planning:

- **El Por Qué (Sprint Goal):** Objetivo realista fijado para el Sprint que comunica el valor del incremento
- **El Qué (PBI Seleccionados):** La selección de historias de usuario o ítems del *Product Backlog* que el equipo se compromete a realizar
- **El Cómo (Plan de Ejecución):** La descomposición en tareas diseñadas por los desarrolladores para crear un Incremento que cumpla con la *Definition of Done*

Su compromiso formal es el **Sprint Goal**. Aunque el objetivo es fijo, el plan de trabajo es flexible: si surgen imprevistos o aprendizajes técnicos durante el Sprint, los desarrolladores pueden renegociar el alcance con el *Product Owner* sin alterar el *Sprint Goal*

Propiedad de los Desarrolladores: Los *Developers* son los únicos responsables de gestionar, adaptar y actualizar el *Sprint Backlog*. Nadie externo puede dictarles cómo transformar los requerimientos en un incremento funcional

Inspección Diaria: Se revisa y ajusta diariamente en la **Daily Scrum** de 15 minutos para medir el progreso hacia el *Sprint Goal*

Naturaleza Temporal: A diferencia del *Product Backlog* (que es un contenedor persistente), el *Sprint Backlog* es temporal

Incremento

Representa un paso concreto e hito tangible hacia el **Product Goal**

Es la suma de todos los elementos del *Product Backlog* completados durante el Sprint, combinado con el valor de los incrementos entregados en iteraciones anteriores

Un equipo puede producir **varios incrementos dentro de un mismo Sprint**. No es necesario esperar a la *Sprint Review* ni al cierre de la iteración para liberar un Incremento

La suma de los incrementos se inspecciona formalmente en la *Sprint Review* para evaluar el progreso real del producto

El **compromiso formal** del Incremento es la **Definición de Hecho (DoD)**

Un ítem del *Product Backlog* se convierte formalmente en parte de un Incremento **en el preciso instante en que satisface la DoD**

Si un ítem no cumple con todos los criterios de la DoD, **no se considera parte del Incremento**, no se contabiliza en la velocidad del equipo, no se puede liberar ni mostrar en la *Sprint Review*, y debe retornar al *Product Backlog* para ser reevaluado

Output Done – Calidad técnica

Representa el **Incremento técnicamente terminado**. Garantiza que el producto fue construido correctamente, cumpliendo los criterios de calidad, integración y pruebas establecidos.

Pregunta: ¿Construimos correctamente el producto?

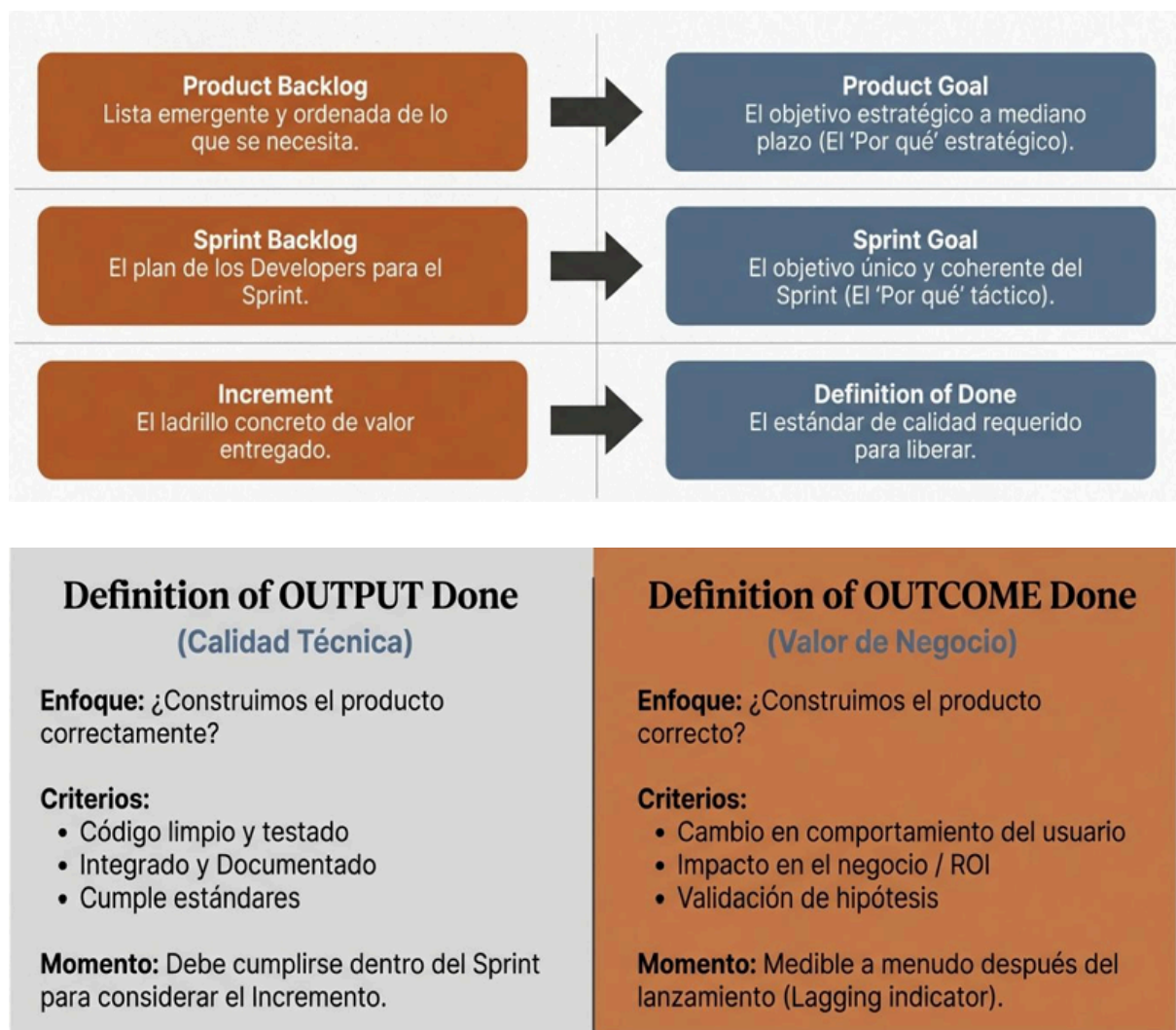
Outcome Done – Valor de negocio

Representa el **resultado o impacto generado por el Incremento**. Permite evaluar si lo desarrollado produjo el valor esperado para el usuario o el negocio.

Pregunta: ¿Construimos el producto correcto? ¿Lo que construimos generó el valor esperado?

Relación

El **Output Done** es necesario para obtener el **Outcome**, ya que primero debemos contar con un Incremento funcional y utilizable para poder medir su impacto.



5. Eventos de Scrum

Los eventos son oportunidades formales para la transparencia, inspección y adaptación. Están diseñados bajo el concepto TimeBox (caja de tiempo con duración máxima prefijada)

con el fin de establecer regularidad, ritmo de trabajo y evitar reuniones no programadas o innecesarias.

El Sprint (El Evento Contenedor)

Contenedor de todos los demás eventos, es el lugar en donde las ideas se convierten en un Incremento de valor utilizable.

Reglas del Evento: Durante el Sprint no se realizan cambios que comprometan el *Sprint Goal*, los estándares de calidad no disminuyen y el alcance de las tareas se puede renegociar entre los *Developers* y el *Product Owner* a medida que se aprende más

Cancelación: Sólo puede ser cancelado por el **Product Owner** si el *Sprint Goal* se vuelve obsoleto por giros del negocio o del mercado

Sprint Planning (Planificación del Sprint)

Tiene como propósito iniciar el Sprint estableciendo colaborativamente el trabajo y la estrategia técnica que se ejecutará en la iteración.

Estructura:

1. **¿Por qué este Sprint es valioso?:** El *PO* propone el valor deseado y el equipo define el *Sprint Goal*.
2. **¿Qué se puede hacer?:** Los desarrolladores seleccionan los ítems (*PBIs*) del *Product Backlog* según su capacidad.
3. **¿Cómo se realizará el trabajo?:** Los desarrolladores desglosan las historias en tareas atómicas formando el plan de ejecución. La unión del *Sprint Goal*, los ítems elegidos y las tareas conforma el **Sprint Backlog**.

Daily Scrum

- **Propósito:** Inspeccionar el avance diario hacia el *Sprint Goal* y adaptar el plan de trabajo del *Sprint Backlog*
- Duración de 15 minutos
- Es un evento exclusivo **por y para los desarrolladores** que se realiza a la misma hora y en el mismo lugar para reducir complejidad
- **No es una reunión de reporte** de estado para la gerencia, sino una sesión para identificar y comunicar impedimentos

Sprint Review (Revisión del Sprint)

- **Propósito:** Inspeccionar el Incremento resultante del Sprint y determinar futuras adaptaciones en el *Product Backlog* junto con los *stakeholder*

- Se concibe como una **sesión de trabajo colaborativa** (evitando reducirla a una simple presentación o demostración pasiva) donde se valida el impacto real y valor de negocio entregado (*Outcome Done*)

Sprint Retrospective (Retrospectiva del Sprint)

- **Propósito:** Inspeccionar cómo se desarrolló el último Sprint en relación con las personas, las interacciones, los procesos, las herramientas y la *Definition of Done*, identificando formas de mejorar en futuros Sprints

Actividad Complementaria: Refinamiento del Product Backlog (*Grooming*)

No está tipificado formalmente como una ceremonia rígida sino como una **actividad continua**, en donde el equipo dedica **tiempo del Sprint** a desglosar, aclarar y estimar historias de usuario futuras para que entren en el Sprint Planning



El **refinamiento del Product Backlog** es la actividad continua mediante la cual el *Product Owner* y los *Developers* descomponen, detallan y redefinen los elementos del *Product Backlog* (PBIs) en piezas más pequeñas, precisas y estimables.

A diferencia de un plan estático tradicional, el *Product Backlog* se concibe como un organismo vivo; el refinamiento actúa como un embudo donde las ideas e hipótesis se transforman progresivamente en elementos de trabajo listos para ser construidos



Triple filtro de Scrum

El **Triple Filtro de Scrum** representa los tres estándares que estructuran la preparación, construcción y validación del trabajo en un proyecto de software.

Se divide entre **un criterio emergente de preparación** (antes de desarrollar) y **dos compromisos formales** (uno técnico y otro de negocio):

1. Definition of Ready (DoR) – El Filtro de Preparación

- **Estatus:** Es una **cualidad emergente** del refinamiento continuo; no constituye un compromiso formal.
- **Artefacto asociado:** Se aplica sobre los ítems del Product Backlog (PBIs).

- **Propósito:** Asegurar que un ítem posee claridad suficiente, un tamaño manejable y dependencias resueltas antes de ser seleccionado para un Sprint. Busca reducir el riesgo y la incertidumbre antes de construir.
 - **Responsabilidad:** Trabajo colaborativo de todo el **Scrum Team** (donde los *Developers* dimensionan y estiman el esfuerzo).
-

2. Definition of Output Done – El Filtro de Calidad Técnica

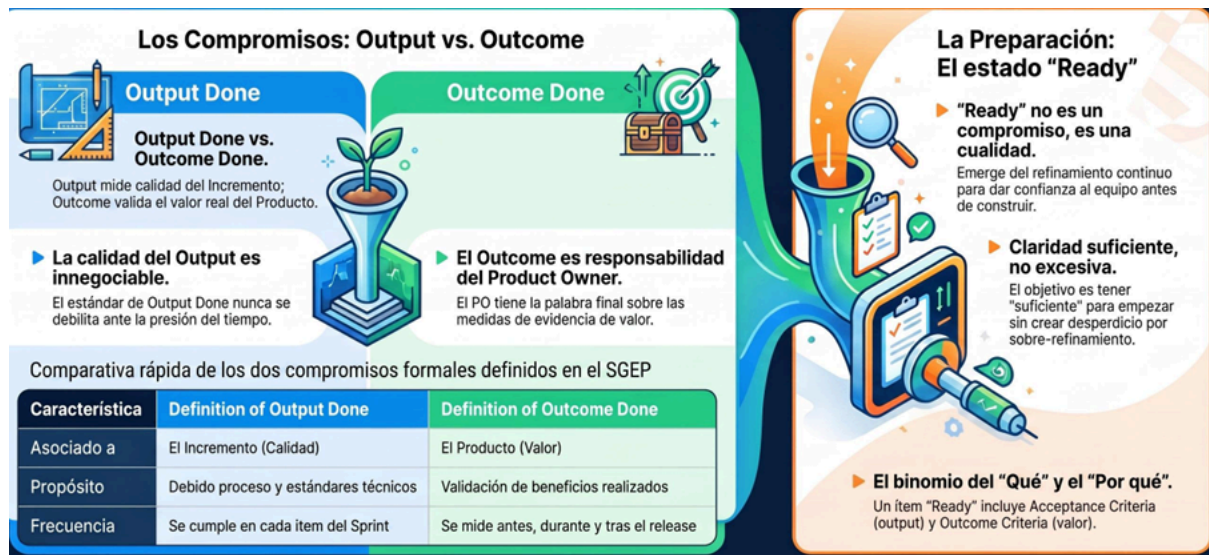
- **Estatus:** Es un **compromiso formal** del proceso.
 - **Artefacto asociado:** Se aplica sobre el **Incremento** (*Increment*).
 - **Propósito:** Garantizar el debido proceso técnico y los estándares de calidad del software. Responde a la pregunta: *¿Construimos el producto correctamente?*. Este estándar es innegociable y jamás se debilita ante la presión de los plazos.
 - **Responsabilidad:** Responsabilidad colectiva de los **Developers**.
-

3. Definition of Outcome Done – El Filtro de Validación de Valor

- **Estatus:** Es un **compromiso formal** centrado en el producto.
 - **Artefacto asociado:** Se aplica sobre el **Producto** (*Product*).
 - **Propósito:** Demostrar y medir mediante evidencias concretas, si la funcionalidad entregada generó un beneficio real. Responde a las preguntas: *¿Construimos el producto correcto?* / *¿Generamos valor real de negocio?*.
 - **Responsabilidad:** Es responsabilidad directa del **Product Owner**, quien posee la palabra final sobre la definición de las métricas de valor.
-

Resumen de la Cadena de Trabajo

- **Ready:** Evalúa si la idea está suficientemente madura para **comenzar a construirse**.
- **Output Done:** Verifica que el software fue **técnicamente bien construido** para poder liberarse.
- **Outcome Done:** Confirma si la liberación del software logró el **éxito y valor real** en el negocio.



SCM

La **Gestión de la Configuración del Software** (*Software Configuration Management - SCM*) es una disciplina de soporte y una actividad transversal que se aplica a lo largo de todo el ciclo de vida del software. Su objetivo es identificar, documentar y controlar los cambios en los artefactos, garantizando la integridad, trazabilidad y visibilidad del sistema.

Definición de ANSI/IEEE: Disciplina que aplica dirección y monitoreo administrativo y técnico a: identificar y documentar las características funcionales y técnicas de los ítems de configuración, controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación y verificar correspondencia con los requerimientos

¿Por qué deberíamos gestionar la configuración del software?

Porque permite establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida

1. Conceptos Clave de SCM

- **Ítem de Configuración de Software:** Es todo aquel artefacto de software que necesita ser compartido, que puede sufrir cambios y sobre el cual se requiere conocer su estado y evolución.
 - **Versión:** Es una instancia particular de un ítem de configuración en un instante o contexto determinado, identificada de forma unívoca.
 - **Variante:** Es una versión alternativa de un ítem de configuración que evoluciona por separado para satisfacer requerimientos específicos (por ejemplo, adaptaciones para distintos sistemas operativos o plataformas).
 - **Configuración del Software:** Representa una "fotografía" o estado instantáneo del conjunto de ítems de configuración con sus respectivas versiones en un momento dado.
 - **Línea Base (Baseline):** Es una configuración o conjunto de ítems que ha sido formalmente revisado y aprobado, sirviendo como punto de referencia estable para desarrollos posteriores.
 - **Repositorio:** Es la base de datos o contenedor que almacena los ítems de configuración, mantiene un historial de cambios, preserva sus relaciones y permite evaluar el impacto de las modificaciones.
 - Repositorio centralizado: un servidor contiene todos los archivos con sus respectivas versiones
 - Repositorio descentralizado: cada usuario tiene una copia exactamente igual al repositorio central, lo que permite otros workflows
 - **Ramas (Branch) y Fusión (Merge):** Las ramas permiten bifurcar la línea principal de desarrollo(master) para experimentar, corregir errores o desarrollar funcionalidades de forma aislada. Posteriormente, los cambios se integran a la rama principal mediante una operación de fusión (*merge*) o simplemente pueden ser descartadas.
-

2. Actividades Fundamentales de la SCM

La disciplina contempla cuatro actividades principales de trabajo:

- **Identificación de Ítems de Configuración:**
 - Identificación unívoca de cada ítem, convenciones de nombrado y versionado y la estructura organizativa del repositorio.

- Clasifica los ítems según su ciclo de vida en ítems de producto, de proyecto o de iteración.



- **Control de Cambios:**

- Regula las modificaciones sobre elementos que forman parte de una línea base.
- Involucra al **Comité de Control de Cambios (CCC)**, el cual realiza un **análisis de impacto** antes de aprobar o rechazar la propuesta.
- Si el cambio es aprobado, se emite una **Orden de Cambio de Ingeniería (OCI)** para proceder con la modificación y su posterior re-incorporación a la línea base.

El Comité de Control de Cambios



Está formado por representantes de todas las áreas involucradas en el desarrollo:

- ❖ Análisis, Diseño
- ❖ Implementación
- ❖ Testing
- ❖ Otros interesados

- **Auditorías de Configuración:**
 - **Auditoría Física de Configuración (PCA):** Verifica que el contenido y la documentación guardados en el repositorio sean consistentes con el código real construido (proceso de *Verificación*).
 - **Auditoría Funcional de Configuración (FCA):** Evalúa de forma independiente que la funcionalidad y desempeño reales del software cumplan con la especificación de requerimientos (proceso de *Validación*).
- **Informes y Registros de Estado:**
 - Mantiene el registro histórico de la evolución del sistema y responde a preguntas sobre el estado de aprobación de un cambio, qué versión implementa una mejora o cuáles son las diferencias entre dos versiones.

Gestión de Configuración de Software en ambientes Ágiles

En los entornos ágiles, la **Gestión de la Configuración del Software (SCM / ACS)** transforma su enfoque tradicional rígido para alinearse con los principios del Manifiesto Ágil. Mientras que el Ágil se enfoca en la dinámica del proyecto y de las personas, la SCM garantiza la **calidad, estabilidad e integridad del producto** a lo largo de su evolución.

1. Filosofía de SCM en Ambientes Ágiles

- **Servir al equipo:** La SCM existe para facilitar el trabajo de los desarrolladores y darles soporte, en lugar de actuar como un mecanismo de control o fiscalización burocrática sobre ellos.

- **Respuesta al cambio:** En lugar de intentar congelar o evitar el cambio (como en el enfoque tradicional mediante comités rígidos), la SCM ágil se diseña para **abrazar y gestionar los cambios frecuentes** de forma segura.
 - **Transparencia y sin fricción:** Se busca eliminar el desperdicio (waste) introduciendo la mayor automatización posible en el flujo de trabajo para que la SCM sea fluida.
 - **Responsabilidad compartida:** La gestión de la configuración deja de ser tarea exclusiva de un departamento o persona aislada y se convierte en una **responsabilidad colectiva de todo el equipo de desarrollo**. Las tareas de SCM se encuentran embebidas en las actividades diarias necesarias para alcanzar el Objetivo del Sprint.
-

2. ¿Qué ocurre con las Actividades Tradicionales de SCM?

- **Comité de Control de Cambios (CCC / CCB):** No existe un comité formal o **burocrático** que evalúe y apruebe cada solicitud individual durante la iteración. En su lugar, el **Product Owner** prioriza y gestiona las solicitudes de cambio a través del *Product Backlog*, permitiendo ajustar el rumbo en cada ciclo.
 - **Auditorías de Configuración:** Las auditorías tradicionales chocan con la cultura de autogestión ágil. La validación de la calidad e integridad se traslada a la **Definition of Done (DoD)**, revisiones entre pares (*peer reviews*) y pruebas automatizadas continuas.
-

3. SCM en Entornos Ágiles vs. Tradicionales

Aspecto	SCM Tradicional	SCM en Entornos Ágiles
Enfoque de Control	Control riguroso mediante procedimientos y comités formales.	Coordinación y seguimiento transparente para servir al equipo.

Comité (CCC/CCB)	Formado formalmente para aprobar/rechazar propuestas de cambio.	No existe un comité burocrático formal; la repriorización la lidera el PO en el backlog.
Auditorías Externas	Actividad formal obligatoria para validar líneas base.	No se aplican auditorías externas rígidas por chocar con la autogestión.
Automatización	Procesos manuales o semicontrolados mediante herramientas tradicionales.	Automatización intensiva mediante Integración Continua (CI) y pruebas automatizadas.
